



UTEC Posgrado

Aprender de los Logs

Observabilidad, memoria de experiencia
y prompts que se reescriben

Desarrollo de Agentes IA

Dr. Vicente Machaca Arceda

UTEC
Universidad
de Ingeniería
y Tecnología

Contenido

El problema de la caja negra

Ver: observabilidad

Las herramientas

Guardar: de logs a memoria

Reescribirse: el prompt como memoria

Casos reales

Práctica

Cierre

Dónde estamos

El problema de la caja negra

Ver: observabilidad

Guardar: de logs a memoria

Reescribirse: el prompt como memoria

Práctica

Cierre

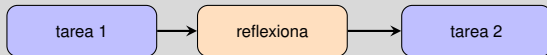
Objetivos

Al final de esta sesión podrás:

- **Instrumentar** un agente y leer sus trazas
- **Elegir** entre LangSmith, LangFuse y logging propio
- **Convertir** logs en memoria útil (ciclo CRUD)
- **Implementar** un agente que reescribe su propio prompt

Venimos de la clase anterior con una deuda pendiente: la corrección se perdía al terminar la tarea.

Retomando: la lección que se evaporaba



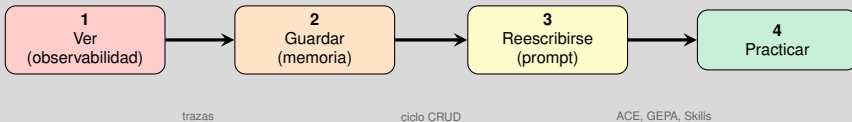
la lección
se perdió

La pregunta de hoy

Cada ejecución deja un rastro. **¿Cómo convertimos ese rastro en una mejora que se quede?**

Pero antes de aprender del rastro, hay que tenerlo. Y muchos agentes no lo tienen.

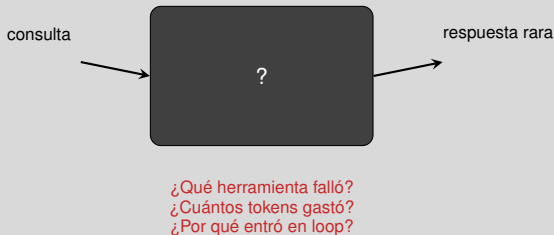
El recorrido de hoy



Cada paso alimenta al siguiente: sin trazas no hay memoria, y sin memoria no hay auto-reescritura.

*Empecemos por el principio: qué pasa cuando **no** puedes ver dentro.*

Un agente sin trazas es una caja negra



No se puede depurar, auditar ni **mejorar** lo que no se puede inspeccionar.

Lo que hace falta tiene nombre y tres componentes.

Discusión en grupo - 3 min

La caja negra que ya tienen

1. Piensen en un sistema que usan hoy: si diera una respuesta rara, ¿podrían saber **qué paso** falló?
2. ¿Qué se está registrando ahora mismo, y quién lo mira?
3. ¿Cuánto tardarían en detectar que el costo por petición se duplicó?

Guarden su respuesta

Volveremos a este sistema en las siguientes discusiones.

Dónde estamos

El problema de la caja negra

Ver: observabilidad
Las herramientas

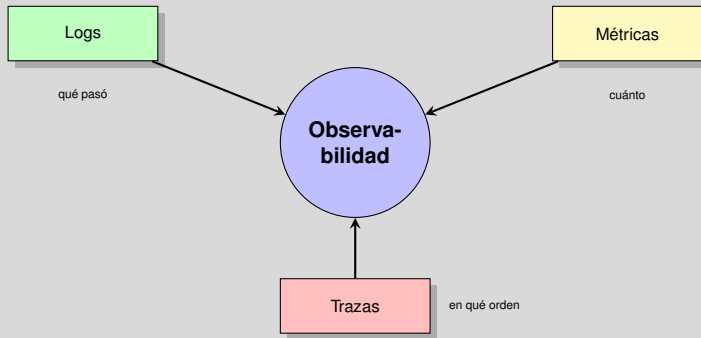
Guardar: de logs a memoria

Reescribirse: el prompt como memoria

Práctica

Cierre

Los tres pilares

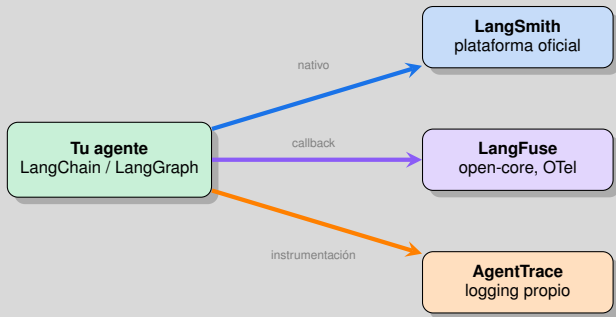


Para agentes, el pilar clave es el tercero: importa el **orden** y el anidamiento de los pasos.

Modelo de datos común a LangSmith y Langfuse; Langfuse lo expone sobre OpenTelemetry.

Veamos cómo se estructura una traza por dentro.

Tres formas de instrumentar



Las veremos una por una, y luego una tabla para decidir.

LangSmith

Opción 1 de 3: la vía rápida.

Qué ofrece

- Tracing, evals, prompts, deployment
- Framework-agnostic (no exige LangChain)
- Dashboards, alertas, colas de anotación

Despliegue

Cloud, híbrido o self-hosted: mismo set de funciones

Ojo con una creencia extendida: ya **no** es solo cloud.

docs.langchain.com/langsmith — planes y despliegue verificados en 2026.

Si el requisito es control total de los datos, la respuesta habitual es la siguiente.

Planes (2026)

Developer	gratis, 5k trazas/mes
Plus	\$39/asiento/mes
Enterprise	a medida

LangFuse

Opción 2 de 3: la vía del control.

Modelo open-core

- Núcleo **MIT**, salvo la carpeta ee/
- Self-host del núcleo: gratis y **sin límite de uso**
- SCIM, audit log, retención: licencia comercial

Sobre OpenTelemetry

Estándar abierto → la misma telemetría puede ir a varios destinos. Sin *vendor lock-in*.

Contexto 2026

Adquirida por **ClickHouse** en enero de 2026; la licencia MIT se mantiene.

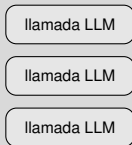
langfuse.com/self-hosting y repositorio MIT (salvo ee/). Adquisición por ClickHouse: enero 2026.

Ambas trazan llamadas al modelo. Pero un agente es más que una lista de llamadas.

AgentTrace: subir un nivel de abstracción

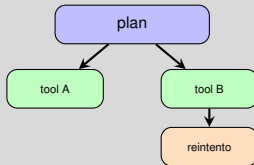
Opción 3 de 3: la vía académica.

Nivel modelo



Lista plana de eventos

Nivel sistema-agente



Grafo de ejecución completo
con relaciones padre-hijo

AgentTrace, arXiv:2602.10133 (2026)

Con las tres sobre la mesa, decidamos.

Recap: cuál elegir

	LangSmith	LangFuse	Propio
Integración LangChain	Nativa	Callback	Manual
Licencia	Propietaria	MIT + ee/	Tuya
Hosting	Cloud/híbrido/self	Cloud + self	Self
OpenTelemetry	Parcial	Nativo	Depende
Costo inicial	Gratis limitado	Generoso	Tu tiempo
Extensibilidad	Limitada	Total	Total

Estrategia híbrida

Desarrollo con LangSmith (feedback rápido) + producción con LangFuse (control de datos).
No son excluyentes.

Tabla propia, contrastada con la documentación oficial de ambas plataformas (2026).

*Ya vemos todo. Ahora la parte que casi nadie hace: **usar** esas trazas para algo más que depurar.*

Discusión en grupo · 3 min

Elijan herramienta para su sistema

1. ¿Sus datos pueden salir de la organización? Eso decide casi todo.
2. ¿Les basta con trazar *llamadas al modelo*, o necesitan el **grafo de ejecución** completo?
3. ¿Quién pagaría la cuenta, y a partir de cuántas trazas al mes duele?

Provocación

Si la respuesta al punto 1 es “no lo sé”, esa es la primera tarea, no elegir plataforma.

Dónde estamos

El problema de la caja negra

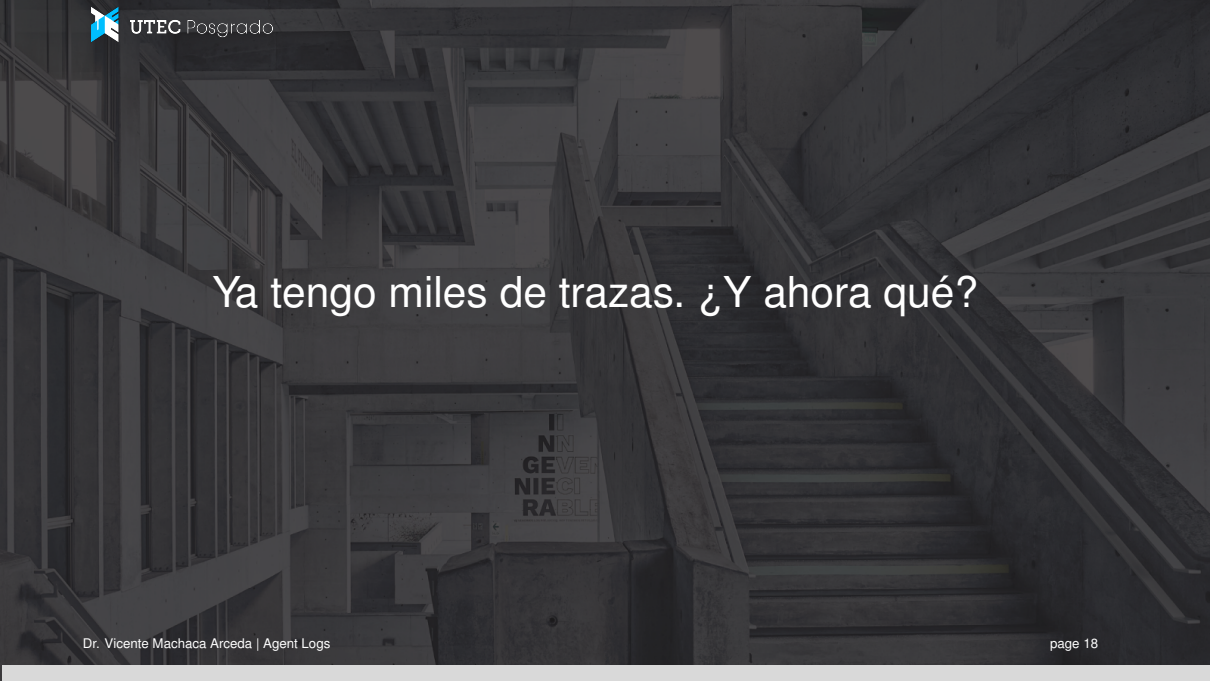
Ver: observabilidad

Guardar: de logs a memoria

Reescribirse: el prompt como memoria

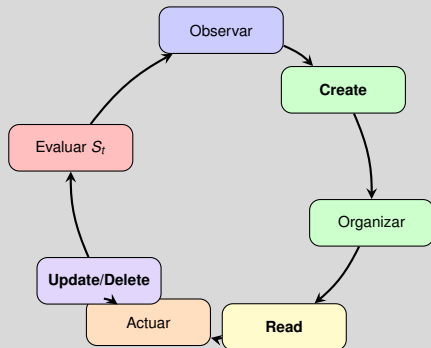
Práctica

Cierre



Ya tengo miles de trazas. ¿Y ahora qué?

El ciclo de memoria guiado por señal



S_t es el **Score** de la traza (éxito, crítica, costo). Es lo que convierte una caché pasiva en un motor de aprendizaje.

Ren et al., arXiv:2607.13104 (2026), §6.2.3 *Memory Processing*.

Cada letra del CRUD esconde un error clásico. Los cuatro:

Los cuatro dilemas del CRUD

Op.	Si te pasas	Si te quedas corto
Create	Ruido en el retrieval	Pierdes capacidad a largo plazo
Read	Distraes al modelo, gastas tokens	Falla la planificación
Update	Fusiones borran detalles	Datos obsoletos persisten
Delete	Pierdes conocimiento crítico	Inundación de ruido

El error más frecuente

Volcar los logs crudos en la memoria. **Create** debe ser *destilación selectiva*, no `append`.

Los cuatro dilemas están enunciados en Ren et al., §6.2.3.

¿Y dónde se guarda lo destilado? La respuesta moderna: en el propio **system prompt**.

Discusión en grupo · 3 min

Diseñen su política de memoria

1. De todo lo que su sistema registra, ¿qué merecería **guardarse** como lección? (**C**)
2. ¿Cuándo se leería esa lección, y cómo evitan inyectarla siempre? (**R**)
3. ¿Qué haría que una lección quede **obsoleta**, y quién la borra? (**U/D**)

Recuerden la clase anterior

Si la lección guardada es falsa, el error se vuelve permanente. Toda regla necesita forma de revocarse.

Dónde estamos

El problema de la caja negra

Ver: observabilidad

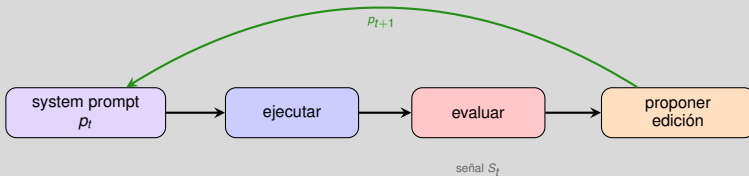
Guardar: de logs a memoria

Reescribirse: el prompt como memoria
Casos reales

Práctica

Cierre

El prompt es código que se puede editar solo



Antes (clase anterior)

Corregíamos **la salida**: la respuesta mejoraba, la instrucción seguía igual.

Ahora

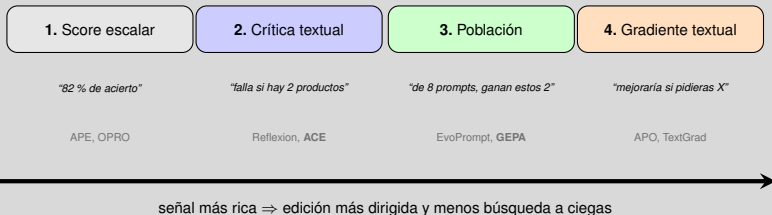
Corregimos **la instrucción**: la mejora se aplica a todas las ejecuciones futuras.

Aquí “prompt” es el **system prompt** estable, el que se reutiliza en cada llamada.

*Lo que distingue a los métodos es **qué información** lleva la señal S_t .*

Cuatro paradigmas, según la riqueza de la señal

Un “paradigma” aquí = qué forma tiene S_t , y por tanto cuánto sabe el sistema sobre *cómo* editar.



Con solo un **número** (1) hay que probar variantes al azar y quedarse con la mejor. Con una **crítica en lenguaje** (2) ya sabes qué corregir. En (3) no editas uno sino que evolucionas **muchos** prompts a la vez. En (4) la crítica viene con **dirección**, imitando un gradiente.

Hoy nos quedamos en el 2. El 3 es el tema de la siguiente clase.

Ren et al., arXiv 2607.13104 (2026), §6.1

El enfoque ingenuo y por qué falla

Paradigma 2: tenemos críticas en lenguaje natural. ¿Cómo las metemos en el prompt?

Intento ingenuo: “dale al LLM el prompt actual más las críticas y pídele que lo **reescriba mejor**”.



Colapso de contexto (*context collapse*)

Al reescribirlo todo de golpe, el LLM **resume** en vez de editar: 18k tokens de conocimiento acumulado se evaporan en 122 genéricos, y el rendimiento cae 10 puntos.

Ejemplo medido en ACE, arXiv:2510.04618.

*La solución no es reescribir mejor: es **dejar de reescribir**. Eso propone ACE.*

ACE: el contexto como *playbook*, no como prompt

ACE = *Agentic Context Engineering*

En vez de tratar el prompt como un texto que se reescribe, lo trata como un **manual de jugadas**: una lista de viñetas cortas e independientes, cada una con una estrategia, una trampa conocida o un formato.

Prompt monolítico

Un bloque de texto.

Actualizarlo = reescribirlo entero.

Riesgo: se pierde lo aprendido.

Playbook de viñetas

Muchas piezas pequeñas.

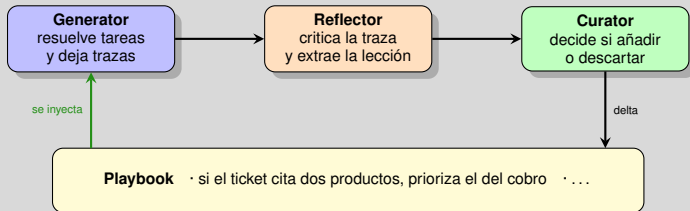
Actualizarlo = **añadir o editar una**.

Lo demás queda intacto.

Es la misma idea que un *changelog* o un commit: cambios **incrementales** y trazables, no versiones nuevas desde cero.

Para producir esas viñetas, ACE reparte el trabajo en tres papeles.

ACE: tres papeles, un playbook



El **Curator** es la pieza clave: aplica un *delta* (añadir, editar o nada) sin tocar el resto del playbook.

vs GEPA (AppWorld)

+12,5 % accuracy, −82,3 % latencia

vs Dynamic Cheatsheet

+7,6 % accuracy, −83,6 % costo

ACE, arXiv:2510.04618. GEPA y Dynamic Cheatsheet son los dos baselines de referencia. Benchmarks: AppWorld (tareas de agente con APIs), FiNER y Formula (razonamiento financiero).

Veámoslo con un prompt de verdad, antes y después.

ACE paso a paso: un ciclo completo

Ejemplo: el clasificador de tickets que implementarán en el notebook.

1. La traza falla

“Olvidé mi password y el link de reset no llega”
predijo **TECNICO** · esperado **CUENTA**

Playbook v_0

Eres un clasificador de tickets de soporte.
Responde: FACTURACION, TECNICO o CUENTA.

2. Reflector extrae la regla

“Si el problema impide **acceder** a la cuenta, es CUENTA aunque el síntoma sea técnico.”

Playbook v_1

Eres un clasificador de tickets de soporte.
Responde: FACTURACION, TECNICO o CUENTA.

Reglas aprendidas:

- Si el problema impide acceder a la cuenta, es CUENTA aunque el síntoma sea tecnico.

3. Curator decide

La compara con el playbook: no está cubierta → **ADD**

Fíjate en lo que **no** pasó: las dos primeras líneas quedaron intactas. Eso es un *delta*.

Repitamos el ciclo unas cuantas veces más y veamos cómo queda.

El playbook después de varios ciclos

Playbook v₃

Eres un clasificador de tickets de soporte.

Responde: FACTURACION, TECNICO o CUENTA.

Reglas aprendidas:

- Si el problema impide acceder a la cuenta, es CUENTA aunque el sintoma sea tecnico.
- Un cargo que el usuario no reconoce es FACTURACION, no CUENTA.
- Si el fallo ocurre solo en un navegador o dispositivo, es TECNICO.

Una candidata rechazada

Reflector propone: *"los problemas de contraseña son de CUENTA"*

Curator: **SKIP** — ya la cubre la regla 1.

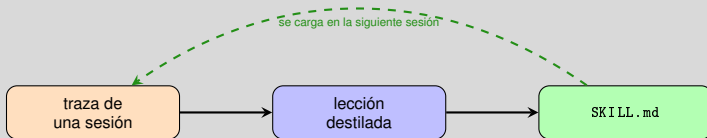
¿Cuándo parar?

Si el Curator responde SKIP casi siempre, el playbook **ya convergió**. Seguir solo añade tokens a cada llamada.

El playbook crece **poco a poco y de forma legible**: puedes leerlo, versionarlo en git y borrar una regla si resulta falsa.

Este patrón no vive solo en el paper: probablemente lo usaste esta semana sin saberlo.

Ya lo estás usando: Skills



Un *skill* es una actualización reutilizable

Se serializa en un archivo, se versiona en git, y es portable entre agentes y sesiones. Es el andamiaje Σ de la clase anterior, hecho texto.

Estándar abierto agentskills.io; concepto de *skill* como actualización reutilizable en Ren et al., §3.2.

Antes de programarlo, veamos un caso real de qué se puede sacar de una traza.

¿Por qué no sirve un debugger normal?

Ya sabemos guardar y reutilizar lecciones. Pero cuando algo falla, hay que **encontrar el paso culpable**.

Programa clásico

línea de código

variable en memoria

barato de re-ejecutar
estado inspeccionable

Agente LLM

step function

evento de entrada/salida

cada paso cuesta dinero
módulos caja negra

Depurar un agente se parece más a depurar un **pipeline de datos** que un programa: las unidades atómicas no son instrucciones, son módulos opacos y **caros de repetir**.

Por eso hace falta una herramienta distinta. LADYBUG es una propuesta académica de cómo sería.

LADYBUG: un debugger para agentes

Qué es

Herramienta interactiva (EDBT 2025) para **trazar, intervenir y re-ejecutar** agentes LLM. Formaliza la ejecución como una traza $T = (e_1, \dots, e_n)$, donde cada $e_i = (p_i, f_i, o_i)$ registra entrada, función y salida de un paso.

Debugger clásico: *imperativo*

Pausas, cambias una variable, sigues. Modificación a modificación, en vivo.

LADYBUG: *declarativo y post-hoc*

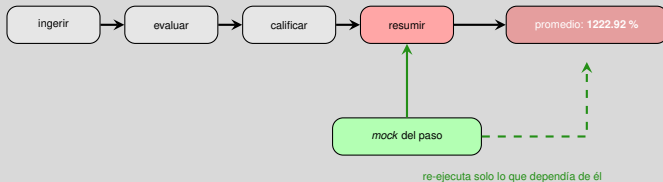
Revisas la traza completa, **encolas** los cambios, y re-ejecuta **solo los pasos afectados**.

Ese matiz existe por el costo: en una traza larga no puedes permitirte re-ejecutar todo tras cada prueba.

Rorseth, Godfrey, Golab, Srivastava & Szlichta, EDBT 2025 (demo). Soporta agentes LlamaIndex; el concepto aplica a LangGraph y CrewAI.

Y cuando ni siquiera tú encuentras el paso culpable, entra un segundo LLM.

El caso del promedio imposible



El problema

Un agente que califica ensayos reporta un promedio de clase de **1222.92 %**. El usuario revisa paso por paso y **no encuentra el error**.

El LLM-aided debugger

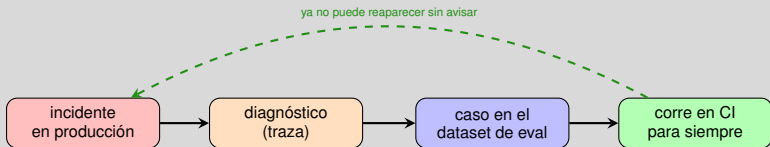
Recibe las firmas y *docstrings* de cada paso más la traza, y debe: (1) decidir si la salida es incorrecta, (2) señalar el **primer** paso donde se rompió, (3) proponer la invocación corregida.

Localizó una alucinación en un paso tardío. Se sustituyó su salida y solo se recalculó lo que dependía de ella.

Fíjate en la ironía: se usa un LLM para arreglar la alucinación de otro LLM.

Un incidente no debería morir en el ticket

Segundo patrón: qué hacer **después** de encontrar el fallo.



Ejemplo: agente en bucle

Síntoma: 20+ llamadas a herramientas, la misma tool repetida, con entradas ambiguas.

El arreglo

Límite de iteraciones · mejores descripciones de tools · detector de patrones circulares.

Sin el último paso, el mismo fallo vuelve en tres meses y nadie recuerda por qué se arregló.

Todo esto asume que puedes loguearlo todo. En producción, no.

Los límites: qué no se loguea y qué se vigila

Privacidad

Nunca loguear PII

Anonimizar IDs

Rotar API keys

Escala

Muestreo en alto volumen

Logging asíncrono

Política de retención

Trazar cuesta: si el logging es síncrono, tu observabilidad se vuelve parte de la latencia que querías medir.

Alertas mínimas para no vivir mirando el dashboard

Error rate $> 5\%$ | Latencia P99 $> 10s$ | Costo diario sobre umbral

Con las reglas claras, vamos al código.

Dónde estamos

El problema de la caja negra

Ver: observabilidad

Guardar: de logs a memoria

Reescribirse: el prompt como memoria

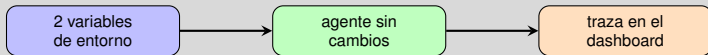
Práctica

Cierre

Práctica: instrumentar y aprender

Primero ver la traza. Después dejar que el agente aprenda de ella.

Agente 1: instrumentado



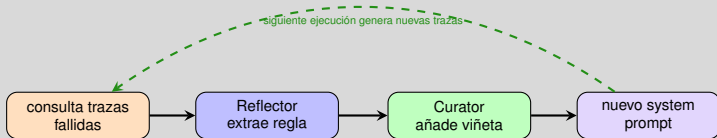
Qué observar

Árbol de llamadas, tokens por nodo, latencia por herramienta, costo total. Es la **anatomía de la traza** que vimos, ahora real.

Ya lo vemos. Ahora hagamos que el agente se lea a sí mismo.

`notebooks/11_logs.ipynb` → Parte 1

Agente 2: mini-ACE



Regla de oro

Añadir viñetas, nunca reescribir el prompt completo. Así evitas el colapso de contexto.

Y aquí acecha la trampa que arruina la mitad de estos experimentos.

`notebooks/11_logs.ipynb` → Parte 2

Experimento guiado

Mide la mejora persistente

1. Corre 10 tareas y guarda el *accuracy* base
2. Deja que el agente destile reglas de sus fallos
3. Corre 10 tareas **nuevas** con el prompt actualizado
4. Fuerza un rewrite completo y compara → ¿reproduces el colapso?

La trampa

Evaluar sobre las mismas tareas que generaron las reglas es *overfitting*, no aprendizaje. Es el hermano del problema del oráculo de la clase anterior.

Cerramos con lo esencial.

Dónde estamos

El problema de la caja negra

Ver: observabilidad

Guardar: de logs a memoria

Reescribirse: el prompt como memoria

Práctica

Cierre

Para llevarse

El hilo de hoy en cuatro frases

- Instrumenta **desde el día 1**, no cuando algo falle
- La traza es materia prima de aprendizaje, no solo de debugging
- Destila; no acumules logs crudos
- Actualiza el prompt con **deltas**, nunca de golpe

Siguiente clase

Hoy un agente mejoró su prompt leyendo su historia. ¿Y si tuviéramos **muchos** agentes, compitiendo y compartiendo lo aprendido?

Referencias I



Ren, Z., Chen, Y., Guo, D., et al. (2026). "Self-Improvements in Modern Agentic Systems: A Survey." arXiv:2607.13104.



Zhang, Q., et al. (2025). "Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models." arXiv:2510.04618.



Agrawal, L., et al. (2026). "GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning." *ICLR 2026*.



"AgentTrace: A Structured Logging Framework for Agent System Observability" (2026). arXiv:2602.10133.



Rorseth, J., et al. (2025). "LADYBUG: an LLM Agent DeBUGger for Data-Driven Applications." *EDBT 2025*.



Yang, C., Wang, X., Lu, Y., et al. (2023). "Large Language Models as Optimizers (OPRO)." arXiv:2309.03409.



Yuksekgonul, M., et al. (2024). "TextGrad: Automatic Differentiation via Text." arXiv:2406.07496.



LangSmith Documentation. <https://docs.langchain.com/langsmith/home>



Langfuse Documentation. <https://langfuse.com/docs>

Preguntas

Preguntas?

Dr. Vicente Machaca Arceda
vmachaca@utec.edu.pe